Beágyazottszoftver-fejlesztés

7. gyakorlat tematikája

A mostani gyakorlat során a STK3700 kártya energiatakarékos módjaival fogunk megismerkedni.

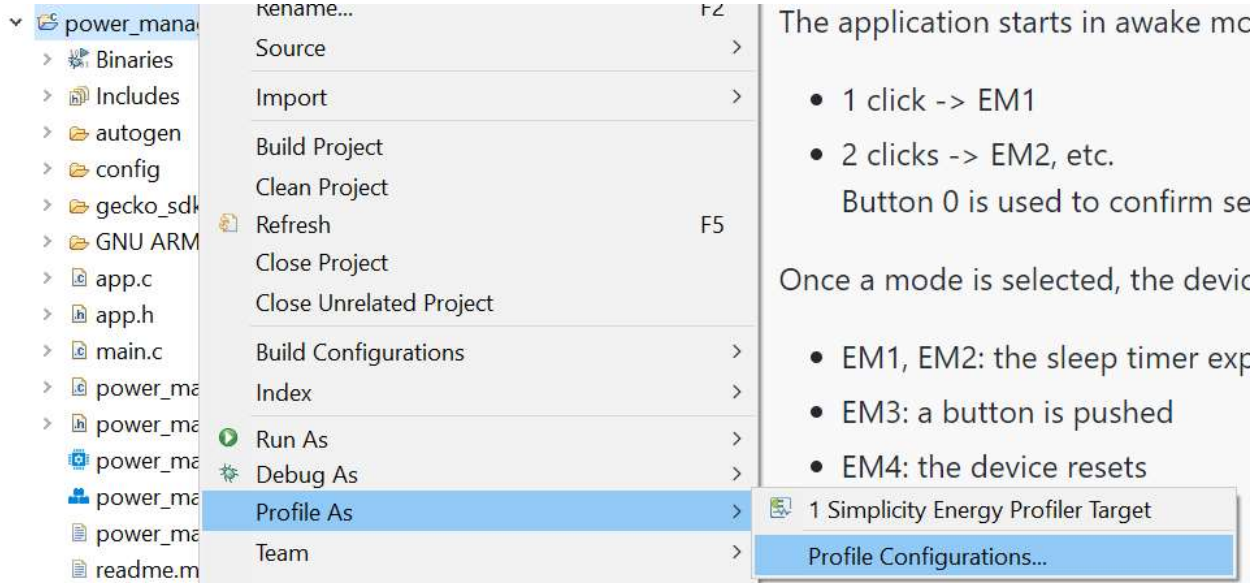
Sajnos az energiatakarékos működés vizsgálatára használt energy profiler el lehetetleníti a debug sorosport használatát a fejlesztő kártyán, így az UART DMA alatti viselkedést nem tudjuk kipróbálni, hanem a folyadékkristályos kijelzőt kell használnunk.

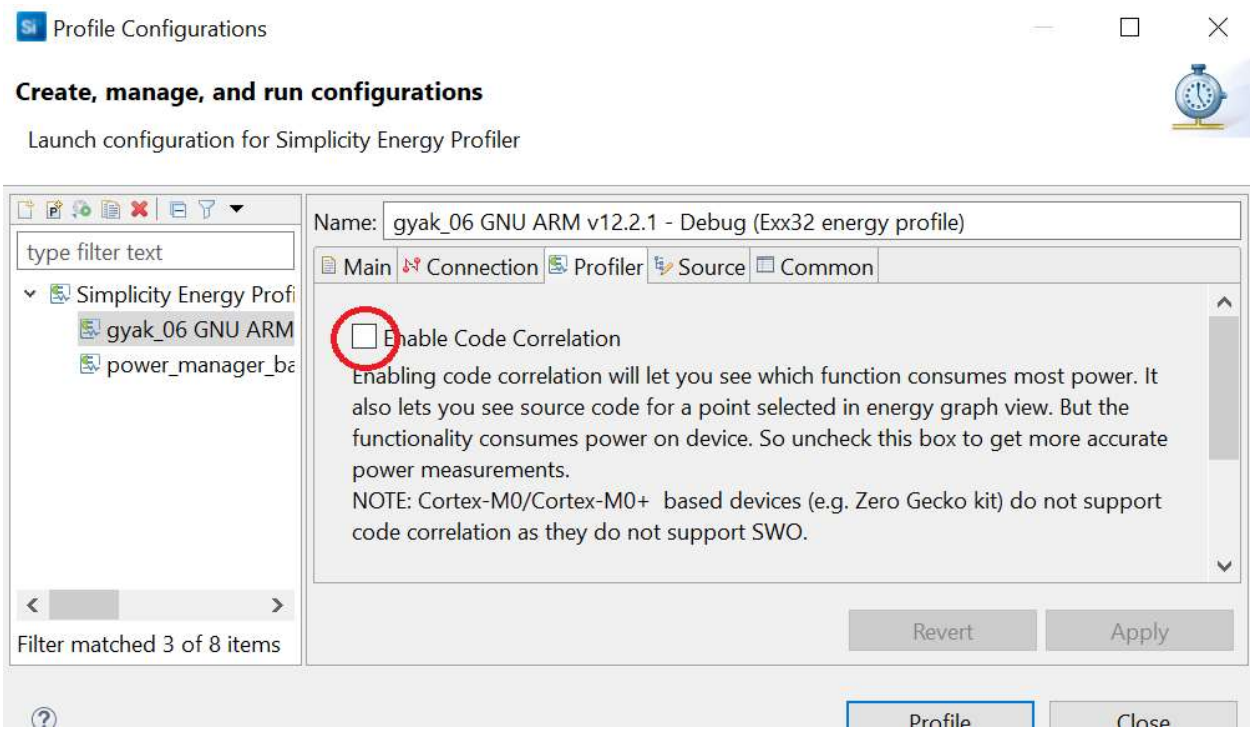
1. Próbáljuk ki a power_manager_baremetal példát

Nyissuk meg a példák közül a power_manager_baremetal, amely két nyomógomb segítségével demonstrálja az EFM32GG990 mikrovezérlő különböző energiatakarékos módjait EM0-EM4-ig.

A mintában a PB1 nyomógomb nyomásainak számával tudjuk kiválasztani az energiatakarékos módot. (1 lenyomás EM1, 2 lenyomás EM2, de a rendszer nem jelzi, hogy hányadik lenyomásnál tartunk), majd az PB0 megnyomásával tudjuk elindítani az adott módot. Az energiatakarékos mód elindulása után a rendszer EM1-EM2 módokban egy timer lejárta után magától visszaáll EM0-ba, EM3-EM4 módokban a PB0 lenyomása, vagy Reset gomb lenyomása szükséges.

Az energy profiler indításánál figyeljünk arra, hogy profiler konfigurációban, a Profiler / Enable Code Correlation opció ki legyen kapcsolva, mert ez olyan debug blokkokat tart aktívan, amik miatt az EM1 alatti viselkedések nem megfigyelhetők. Figyeljünk arra is, hogy a rendszer néha szereti ezt az opciót magától visszakapcsolni. A profile-al való futtatás esetén néha szükséges a letöltés után egy reset-et nyomni a board-on, hogy valóban elinduljon a project.





2. Saját energiatakarékos alkalmazás az LCD használatával

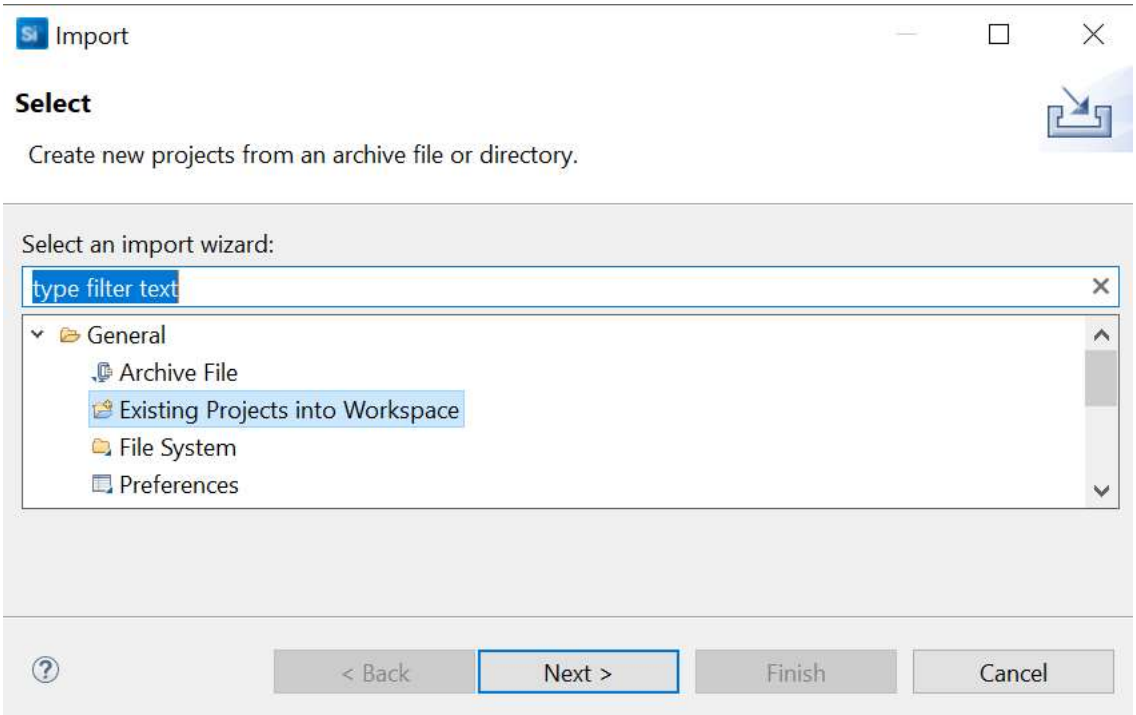
2.1. Kiinduló project importálása

A feladat kiindulási pontja az előző alkalommal használt LCD vezérlős alkalmazás lesz. Ezt fogjuk tovább bővíteni.

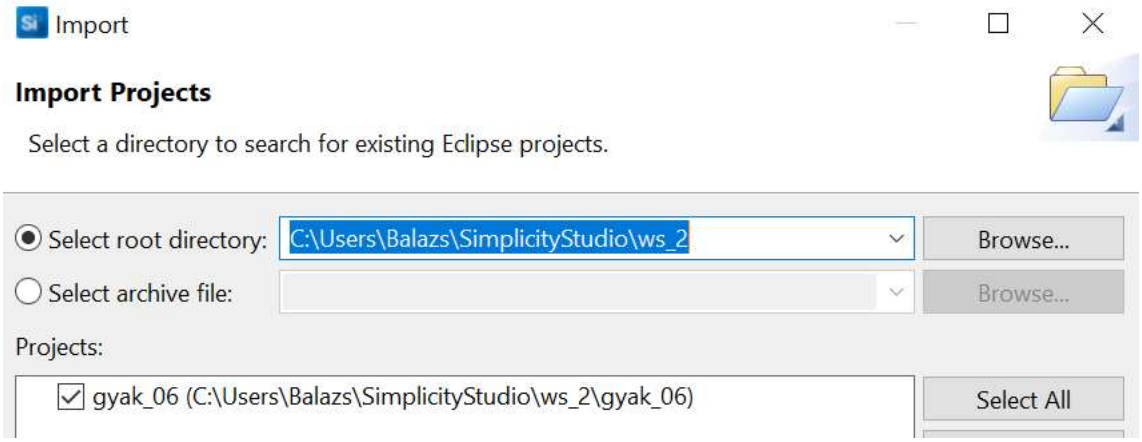
A honlapról letöltött project importálása:

1, töltsük is csomagoljuk ki a projectet a Workspace könyvtárunkba

2, importáljuk be a projectet: jobb gomb a project fülön: Import/General/Existing Project ….

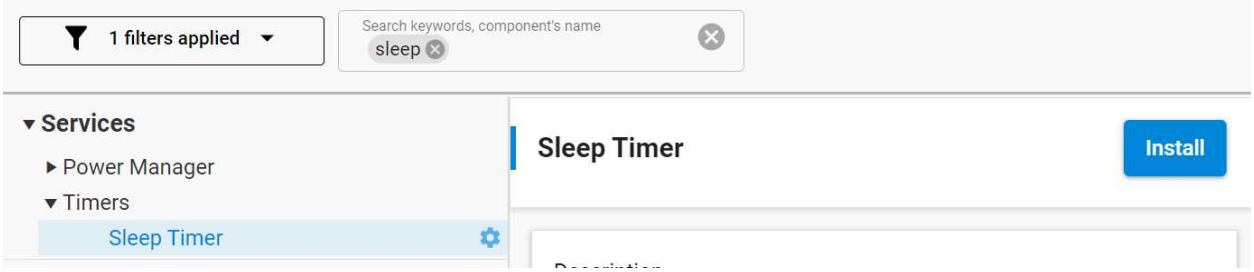


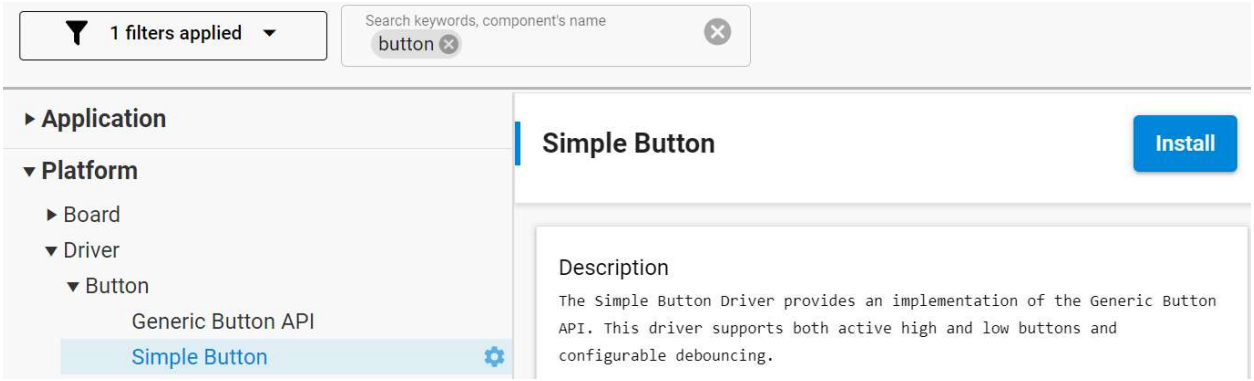
Figyeljünk arra, hogy az importálásnál a file kereső a Workspace directory-n álljon! NE menjünk bele az adott project könyvtárjába! Igy válasszuk ki a kívánt projectet. Ha más SDK verziónk van, mint amit minta project használ, az gondot okozhat, ilyenkor a Laucher fül manage SDK opciójával lehet próbálkozni.



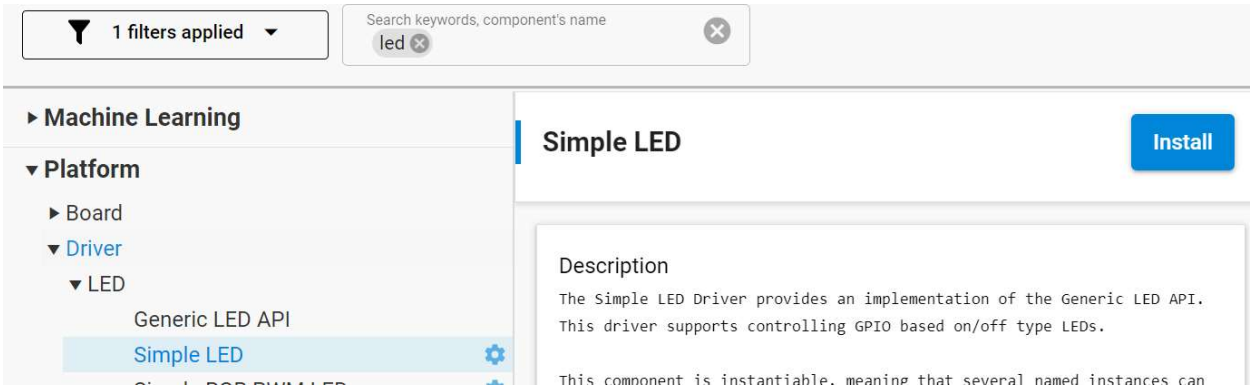
2.2. Timer, nyomógombok, LED-ek hozzáadása

A szokásos módon az .slcp file segítségével adjuk hozzá az alábbi software komponenseket.





Mind a BT0, mind a BT1 instance-át.



Elég a LED0 instance, de ahogy akarjátok …

2.3. Periodikus kijelzés az LCD-n

Hozzunk létre egy periodikusan futó Sleep Timer-t, ami növeli a kijelzett értéket az LCD kijelzőn. Sleep timer dokumentáció a forrás file-ból, vagy:

https://docs.silabs.com/gecko-platform/4.4.6/platform-service/sleeptimer

A szám kijelzése nyugodtan történhet az app_process_action()-ben, nem kell átrakni a megszakításból hívódó callback-be, ez később sem fog gondot okozni.

Az **app.c** fájl így módosult az új részlet hozzáadásával.

```
/**************************************************************************//**
 * Initialize application.
 *****************************************************************************/
```

```c
#include "segmentlcd.h"
#include "sl_sleeptimer.h"

volatile int32_t app_lower_number = 0;
sl_sleeptimer_timer_handle_t timer;

static void app_timeout_callback(sl_sleeptimer_timer_handle_t *handle, void *data);

void app_init(void)
{
  /* Enable LCD without voltage boost */
  SegmentLCD_Init(false);
  /* Init Sleep Timer */
  sl_sleeptimer_init();

  // Create timer for waking up the system periodically.
  sl_sleeptimer_start_periodic_timer_ms(&timer,
                                        500,
                                        app_timeout_callback, NULL,
                                        0,

SL_SLEEPTIMER_NO_HIGH_PRECISION_HF_CLOCKS_REQUIRED_FLAG);


}

/***************************************************************************//**
 * App ticking function.
 ******************************************************************************/
void app_process_action(void)
{
  SegmentLCD_LowerNumber(app_lower_number);
}

void app_timeout_callback(sl_sleeptimer_timer_handle_t *handle, void *data)
{
  app_lower_number++;
}
```

Próbáljuk ki a program futását a Profile lehetőség használatával.

2.4. Periodikus kijelzés az LCD-n, nyomógomb hatására világító LED-el

Hozzunk létre egy nyomógomb callback függvényt és használjuk a nyomógombok callback-et a demó kártya LED-jének ki/be kapcsolására:

https://docs.silabs.com/gecko-platform/4.4.6/platform-driver/button

https://docs.silabs.com/gecko-platform/4.4.6/platform-driver/simple-led

A nyomógombok és led-ek, ha megtartjuk az alap project struktúrát automatikusan inicializálódnak. Íme ez a részlet (ha valaki akarja ezekből a függvényekből könnyen ki lehet szedni a szükséges példány hivatkozásokat is):

```
void sl_driver_init(void)
{
  GPIOINT_Init();
  sl_simple_button_init_instances();
  sl_simple_led_init_instances();
}
```

Az **app.c** fájl így módosult az új részlet hozzáadásával.

```
/***************************************************************************//**
 * Initialize application.
 ******************************************************************************/
#include "segmentlcd.h"
#include "sl_sleeptimer.h"
#include "sl_simple_led.h"
#include "sl_simple_led_instances.h"
#include "sl_simple_button.h"
#include "sl_simple_button_instances.h"

volatile int32_t app_lower_number = 0;
sl_sleeptimer_timer_handle_t timer;

static void app_timeout_callback(sl_sleeptimer_timer_handle_t *handle, void *data);

void app_init(void)
{
  /* Enable LCD without voltage boost */
  SegmentLCD_Init(false);
  /* Init Sleep Timer */
  sl_sleeptimer_init();

  // Create timer for waking up the system periodically.
  sl_sleeptimer_start_periodic_timer_ms(&timer,
                                        500,
                                        app_timeout_callback, NULL,
                                        0,

SL_SLEEPTIMER_NO_HIGH_PRECISION_HF_CLOCKS_REQUIRED_FLAG);

 }

/***************************************************************************//**
 * App ticking function.
 ******************************************************************************/
void app_process_action(void)
{
  SegmentLCD_LowerNumber(app_lower_number);
}

void app_timeout_callback(sl_sleeptimer_timer_handle_t *handle, void *data)
{
  app_lower_number++;
```

```
}

void sl_button_on_change(const sl_button_t *handle)
{
  if (&sl_button_btn0 == handle)
  {
    if (sl_button_get_state(handle) == SL_SIMPLE_BUTTON_PRESSED)
      {
        sl_led_turn_on(sl_simple_led_array[0]);
      }
    else
      {
        sl_led_turn_off(sl_simple_led_array[0]);
      }
  }
}
```

2.5. Órajelfrekvencia hatásának vizsgálata

A processzor frekvencia változtatásának a hatása. Próbáljuk ki, hogy hogyan viselkedik a fogyasztás, hogyha megváltoztatjuk a processzor frekvenciáját.

A processzor frekvencia a megváltoztatásának a legegyszerűbb módja, hogyha nem a PLL-t állítgatjuk, hanem a processzor órajel forrását átkapcsoljuk a belső RC oszcillátorra, és annak az egyszerű függvényeivel állítjuk be a tényleges működési frekvenciát. Ehhez a

- Órajelforrás inicializációja az alábbi függvényben található:
  - `sl_status_t sl_device_init_clocks(void)`
- Az eredeti `CMU_CLOCK_SELECT_SET` et ki kell cserélni az alábbira, és beállítani egy új frekvenciát:
  - `CMU_CLOCK_SELECT_SET(HF, HFRCO);`
  - `CMU_HFRCOBandSet(cmuHFRCOBand_1MHz);`

Érdemes a CMU_CLOCK_SELECT_SET(LFA, LFRCO); CMU_CLOCK_SELECT_SET(LFB, LFRCO); függvényeket is a fenti módon átírni, hogy az alacsony frekvenciás órajelek a magas frekvenciás kristálytól függetlenül működjenek.

A módosult **`sl_device_init_clocks`** függvény:

```
sl_status_t sl_device_init_clocks(void)
{
 // CMU_CLOCK_SELECT_SET(HF, HFXO);
  CMU_ClockEnable(cmuClock_HFLE, true);

  CMU_CLOCK_SELECT_SET(HF, HFRCO);
  CMU_HFRCOBandSet(cmuHFRCOBand_21MHz);

  CMU_CLOCK_SELECT_SET(LFA, LFRCO);
  CMU_CLOCK_SELECT_SET(LFB, LFRCO);

……
```

2.6. Energiatakarékos módok kipróbálása

A Silicon Labs-os környezetben az energiatakarékos módokat két absztrakciós szinten használhatjuk szoftverből. Az egyszerűbb alacsony szintű használat pár függvény hívást igényel, ugyanakkor komplex feladatoknál szükség lehet a Power Manager Driver funkcióra. Mi az egyszerűbb alacsony szintű megoldást fogjuk most bemutatni, aki szeretné megérteni a magasabb absztrakció szintű Power Manager Drivert, az power_manager_baremetal példa projectben talál egy jó bemutatást annak a használatára.

Az energiatakarékossági funkciók használatához az

```
#include "em_emu.h"
```

header file-ra van szükségünk. A felhasználható függvények a:

```
void EMU_EnterEM1(void);
void EMU_EnterEM2(bool restore);
void EMU_EnterEM3(bool restore);
```

Ezek a függvények közül nálunk csak az EM1-t és EM2-t aktiváló használható, az alkalmazásunk periféria kötöttségei miatt. Az EM2 függvényeknél érdemes a restore flag-et true-ra állítani, hogy helyreállítsa a felébredés után az órajeleket.

Az **app.c** fájl módosult részlete:

```
void app_process_action(void)
{
  SegmentLCD_LowerNumber(app_lower_number);
  EMU_EnterEM1();
}
```

Érdemes megnézni, hogy elég ezt az egy sort lerakni a programba, hiszen ez a sor úgy néz ki, hogy amikor meghívjuk leállítja a processzort, és a külső esemény: Sleep Timer megszakítása, vagy a nyomógomb megszakítása automatikusan felébreszti a processzort és innen folytatódik a végrehajtás.

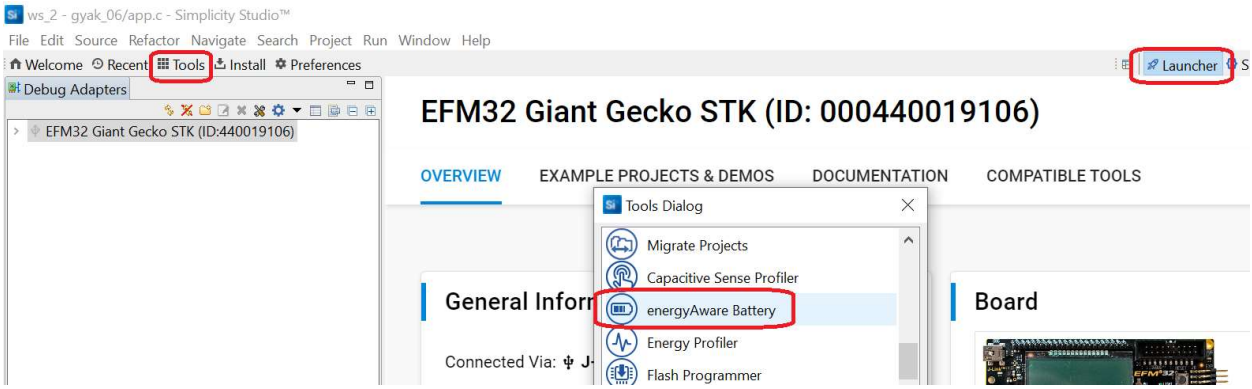
Próbáljuk ki a működést az EMU_EnterEM2(true)-val is.

Nézzük meg, hogy milyen minimális az LCD kijelző fogyasztása (hasonlítsuk össze az EM2 fogyasztást a gyári példa EM2 fogyasztásával). Nézzük meg a LED bekapcsolásának a hatását!

Opcionális feladat: Próbáljuk ki, hogy hogyan változik a fogyasztás, hogyha csak akkor jelenítjük meg az információt a kijelzőn, hogyha a PB1 nyomógombot megnyomjuk. (Látható, hogy alig változik a fogyasztás, mert a statikusan bebillentett Reflective szegmenseknek szinte nincs fogyasztásuk), a nyomógomb 1Mohm-os felhúzó ellenállásának is látványosabb a hatása.

2.7. Battery estimator kipróbálása

A Simplicity Studio-ban van egy elég szofisztikát elem fogyasztás kalkulátor. Ezt a Laucher fül Tools opciója alatt lehet elérni.



Próbáljuk ki ezt a kalkulátort: Hozzunk létre egy alkalmazást ami két AA ceruzaelemről jár. Van neki egy aktív állapota, amikor pár interfészt I2C, UART, ADC stb. bekapcsolunk az aktívan futó processzor mellett. Tartson ez az aktív állapot mondjuk 1s-ig, és hozzunk létre egy EM2 állapotot is, aktív RTC-vel, ami mondjuk 59s-ig tart. Nézzük meg az eredményeket, játszunk el egy kicsit azzal, hogy minek mi a hatása.




eABattery
File Simulation Help
Untitled1 - Giant Gecko
Untitled1 - 1x2 Alkaline AA
Untitled1 - 1x2 Alkaline AA
Untitled1 - 1x2 Alkaline AA
MCU
Average current: 116,674µA
Estimated system operating time: 2 y 222 d
Battery
Rated capacity: 3,000000AH
Untitled1 - 1x2 Alkaline AA
3
2,5
2
Minimum operating voltage